

https://github.com/jambao24/VineMapper-jambao24/blob/main/projects/East_Southeast_Asian_Groups_per_county/

using the original as a template

https://github.com/winstonhoyle/VineMapper/blob/main/projects/ethnicity/East_Asian_Groups_Per_County/FormatData.ipynb

2026.01.09- downloading cb_2024_us_county_500k.zip from here

<https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000>

actually this may be more accurate? <https://www2.census.gov/geo/tiger/GENZ2024/shp/>

```
import requests
import geopandas as gpd
import pandas as pd
```

###Open County data

```
from osgeo import gdal
gdal.SetConfigOption('SHAPE_RESTORE_SHX', 'YES')

# https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-subdivisor
# https://stackoverflow.com/questions/61436956/set-shape-restore-shx-config-option-to-yes
# requires both .shp and .shx files in file directory
file_path = "/content/cb_2024_us_county_500k.shp"
counties_gdf = gpd.read_file(file_path)
```

Get Ethnic Data

```
import json

r = requests.get("https://api.census.gov/data/2023/acs/acs5/groups/B02018.json")
columns_obj = r.json()
# uploading saved B02018.json file since Census API isn't working on 2026.01.14
#with open('B02018.json', 'r') as f:
#    columns_obj = json.load(f)
```

```
###Get columns to query and rename for later
columns = []
rename_vars = {}
variables = columns_obj["variables"]
```

```
for name, variable in list(variables.items()):
    v_split = variable["label"].split("!!")
    if len(v_split) < 3:
        continue

    if v_split[0] == "Estimate":
        label = v_split[-1]
        rename_vars[name] = label

    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "East Asian:":
        columns.append(name)

    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "Southeast Asian:":
        columns.append(name)
```

```
columns.append("GEO_ID")
columns_formatted = ",".join(columns)

print(columns)
print(columns_formatted)

# Census API isn't working... will find a workaround

# generated Census API key: a2944e73e2d423d65f6ced945a26fd13b3817ef4
# let's see if this works
response = requests.get(
    f"https://api.census.gov/data/2023/acs/acs5?get={columns_formatted}&for=county:*&key=a2944e73e2d423d65f6ced945a26fd13b3817ef4"
)

# https://api.census.gov/data/2023/acs/acs5?get={B02018_016M,B02018_017E,B02018_017M,B02018_018E,B02018_014M,B02018_015E,B02018_016M,B02018_017E,B02018_017M,B02018_018E,B02018_014M,B02018_015E,B02018_015M,B02018_016M}
```

```
# test run
response_TW = requests.get(
    f"https://api.census.gov/data/2023/acs/acs5?get={'B02018_008E','B02018_008M'}&for=coi
)
```

```
# redundant if I'm just importing formmted_df.csv directly
data = response.json()
columns = data[0]
rows = data[1:]
df = pd.DataFrame(rows, columns=columns)

print(df.columns)
print(df.head)
```

```

    'B02018_008M', 'B02018_009E', 'B02018_009M', 'B02018_006M',
    'B02018_007E', 'B02018_007M', 'B02018_008E', 'B02018_002E',
    'B02018_020E', 'B02018_020M', 'GEO_ID', 'state', 'county'],
dtype='object')
<bound method NDFrame.head of      B02018_016M B02018_017E B02018_017M B02018_018E B0201
0          31          0          31          36          31          0
1          31          0          31         307         170         52
2          25          0          25          0          25          0
3          25          0          25          0          25          0
4          31          0          31          0          31          0
...         ...         ...         ...         ...         ...
3217        35          0          35          0          35          0
3218        21          0          21          0          21          0
3219        28          0          28          0          28          0
3220        28          0          28          0          28          0
3221        32          0          32          0          32          0

      B02018_015M B02018_016E B02018_018M B02018_019E ... B02018_006M \
0          31          0          60          0 ...          31
1          74          0         257         240 ...          31
2          25          0          25          5 ...          25
3          25          0          25          0 ...          25
4          31          0          31          0 ...          31
...         ...         ...         ...         ... ...         ...
3217        35          0          35          0 ...          35
3218        21          0          21          0 ...          21
3219        28          0          28          0 ...          28
3220        28          0          28          0 ...          28
3221        32          0          32          0 ...          32

      B02018_007E B02018_007M B02018_008E B02018_002E B02018_020E B02018_020M \
0          0          31          0         199          0          31
1          0          31          6         640          0          31
2          0          25          0          47          0          25
3          0          25          0          0          0          25
4          0          31          5          1          0          31
...         ...         ...         ...         ...         ...
3217        0          35          0          0          0          35
3218        0          21          0          0          0          21
3219        0          28          0          0          0          28
3220        0          28          0          0          0          28
3221        0          32          0          0          0          32

      GEO_ID state county
0    0500000US01001    01    001
1    0500000US01003    01    003
2    0500000US01005    01    005
3    0500000US01007    01    007
4    0500000US01009    01    009
...         ...         ...
3217  0500000US72145    72   145
3218  0500000US72147    72   147
3219  0500000US72149    72   149
3220  0500000US72151    72   151
3221  0500000US72153    72   153

```

```
[3222 rows x 41 columns]>
```

```
## import df from csv since API responses aren't working in this instance
```

```
#df = pd.read_csv("formtted_df.csv")
## observation- GEOIDFQ is 0500000US02170 but GEOID is 02170
## just create a duplicate column of GEOIDFQ in df, remove everything before
#df["GEO_ID"] = df["GEOIDFQ"]
#df["GEO_ID"] = df["GEO_ID"].str.replace("0500000US", "")
##df = df.rename(columns={"GEOID": "GEOIDFQ"})
```

```
estimate_cols = [col for col in df.columns if col.endswith("E")]
print(estimate_cols)
```

```
formtted_df = df[["GEO_ID", *estimate_cols]]
formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)
```

```
# https://www.geeksforgeeks.org/pandas/pandas-combine-columns/
# add Taiwanese to Chinese, then delete Taiwanese column
formtted_df['B02018_002E'] += formtted_df['B02018_008E']
formtted_df = formtted_df.drop(columns=['B02018_008E'])
```

```
estimate_cols.remove('B02018_008E')
```

```
formtted_df["most_common_ancestry_raw"] = formtted_df[estimate_cols].idxmax
```

```
print("formtted_df.columns after merging Taiwanese")
print(formtted_df.columns)
#print(formtted_df.most_common_ancestry_raw)
```

```
['B02018_017E', 'B02018_018E', 'B02018_015E', 'B02018_016E', 'B02018_019E', 'B02018_012E',
formtted_df.columns after merging Taiwanese
```

```
Index(['GEO_ID', 'B02018_017E', 'B02018_018E', 'B02018_015E', 'B02018_016E',
      'B02018_019E', 'B02018_012E', 'B02018_013E', 'B02018_014E',
      'B02018_010E', 'B02018_011E', 'B02018_005E', 'B02018_006E',
      'B02018_003E', 'B02018_004E', 'B02018_009E', 'B02018_007E',
      'B02018_002E', 'B02018_020E', 'most_common_ancestry_raw'],
      dtype='object')
```

```
/tmp/ipython-input-3137767251.py:14: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/10min.html
formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)
```

```
/tmp/ipython-input-3137767251.py:20: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/10min.html
formtted_df['B02018_002E'] += formtted_df['B02018_008E']
```

```
# https://www.geeksforgeeks.org/pandas/pandas-combine-columns/
```

```
# update "most_common_ancestry_raw" to B02018_002E "Chinese" for all instances where it is not Chinese
formtted_df['most_common_ancestry_raw'] = formtted_df['most_common_ancestry_raw'].replace('Chinese', B02018_002E)
```

```
print(formtted_df.columns)
print(formtted_df.head())
```

```

3219  0500000US72149      0      0      0      0
3220  0500000US72151      0      0      0      0
3221  0500000US72153      0      0      0      0

```

```

      B02018_019E  B02018_012E  B02018_013E  B02018_014E  B02018_010E  \
0              0          305          0          0          0
1            240         1086         170         137          0
2              5           7           0           0          0
3              0           0           0           0          0
4              0          246           0           0          0
...          ...          ...          ...          ...          ...
3217           0           0           0           0          0
3218           0           0           0           0          0
3219           0           0           0           0          0
3220           0          11           0           0          0
3221           0           0           0           0          0

```

```

      B02018_011E  B02018_005E  B02018_006E  B02018_003E  B02018_004E  \
0              0          160           0           0          233
1              0          303           0           0          398
2              0           48           0           0          16
3              0           23           0           0           0
4              0           7           0           0           0
...          ...          ...          ...          ...          ...
3217           0           0           0           0          0
3218           0           0           0           0          0
3219           0           0           0           0          0
3220           0           0           0           0          0
3221           0           0           0           0          0

```

```

      B02018_009E  B02018_007E  B02018_002E  B02018_020E  \
0              0           0          199           0
1              0           0          646           0
2              0           0           47           0
3              0           0           0           0
4              0           0           6           0
...          ...          ...          ...          ...

```

```
3219          B02018_017E
3220          B02018_012E
3221          B02018_017E
```

```
[3222 rows x 20 columns]>
```

```
def check_margin_error(row) -> str:
    geo_id = row["GEO_ID"]
    ethnicity_col = row["most_common_ancestry_raw"]
    val = row[ethnicity_col]

    if not val:
        return None

    moe_col = ethnicity_col.replace("E", "M")

    # Check if the margin of error column exists in df
    if moe_col not in df.columns:
        # If the MOE column is missing, we cannot calculate rmoe_val, so return None
        # This handles cases where formtted_df.csv might not contain 'M' columns.
        return None

    moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])

    # Avoid division by zero if val is 0
    if val == 0:
        return None

    rmoe_val = abs(moe_val / val)
    if rmoe_val < 0.50:
        return variables[ethnicity_col]["label"].split("!!")[-1]
    else:
        return None
```

```
formtted_df["most_common_ancestry"] = formtted_df.apply(
    lambda row: check_margin_error(row), axis=1
)
```

```
/tmp/ipython-input-3274153329.py:17: FutureWarning: Calling int on a single element Series
moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])
```

```
rename_vars["GEO_ID"] = "GEODIDFQ"
formtted_df = formtted_df.rename(columns=rename_vars)

#formtted_df = formtted_df.rename(columns={"B02018_002E": "Ethnic Chinese"
formtted_df = formtted_df.rename(columns={"Chinese, except Taiwanese": "Et
# https://sparkbyexamples.com/pandas/pandas-replace-values-based-on-conditions/
formtted_df['most_common_ancestry'] = formtted_df['most_common_ancestry'].

print(formtted_df.columns)
#print(formtted_df.head)
```

```
#print(formtted_df.head)
```

```
...
```

```
# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas :  
from google.colab import files  
formtted_df.to_csv('formtted_df.csv')  
files.download('formtted_df.csv')  
...
```

```
Index(['GEOIDFQ', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',  
      'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',  
      'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',  
      'Ethnic Chinese', 'Other Southeast Asian', 'most_common_ancestry_raw',  
      'most_common_ancestry'],  
      dtype='object')  
'\n# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas-dataframe-i  
n-google-colab\nfrom google.colab import files\nformtted_df.to_csv('formtted_df.csv')\nfiles.download('formtted_df.csv')\n'
```

[https://www.arcgis.com/apps/mapviewer/index.html?](https://www.arcgis.com/apps/mapviewer/index.html?url=https://geo.dot.gov/server/rest/services/Hosted/County_cb_2018_us_state_500k/FeatureServer&source=sd)

[url=https://geo.dot.gov/server/rest/services/Hosted/County_cb_2018_us_state_500k/FeatureServer&source=sd](https://geo.dot.gov/server/rest/services/Hosted/County_cb_2018_us_state_500k/FeatureServer&source=sd) this is pretty cool

<https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000>

2026.01.09 <https://pdxedu.maps.arcgis.com/apps/mapviewer/index.html> using 2019 login info to access ArcGIS and play with shp file... runtime issue in Google Colab is with the Key in the file I have here...

KeyError: 'AFFGEOIDFQ'

just used the .dbf spreadsheet from the zip file that contains GEO_ID columns, reran that code snippet and got the following error:

ValueError: Cannot transform naive geometries. Please set a crs on the object first.

<https://stackoverflow.com/questions/64421284/geopandas-valueerror-cannot-transform-naive-geometries-please-set-a-crs-on-t>

```
###Merge Data
```

```
#print(counties_gdf)  
# https://geopandas.org/en/stable/docs/user\_guide/io.html  
counties_gdf_xls = gpd.read_file("cb_2024_us_county_500k.dbf")  
#print(counties_gdf_xls.columns)  
#print(formtted_df.columns)  
  
gdf = counties_gdf_xls.merge(formtted_df, on="GEOIDFQ", how="inner")
```



```
print("post merge:")
print(gdf.columns)
#print(gdf.head())

'''
#gdf.set_crs('epsg:3857')
gdf = gdf.to_crs(9311)
'''

#https://stackoverflow.com/questions/11250870/sqlite3-open-unable-to-open-
#sqlite3_open_v2("data/EastSoutheast_Asian_Groups_Per_County.gpkg", &db, S

#https://gis.stackexchange.com/questions/298530/how-do-i-write-a-geopandas
gdf.to_file("output.json", driver="GeoJSON")
#gdf.to_file("data/EastSoutheast_Asian_Groups_Per_County.gpkg")

gdf.groupby("most_common_ancestry").size().reset_index(name="COUNT").sort_
"COUNT", ascending=False
)

#print(gdf[['NAME', 'STATE_NAME', 'GEOIDFQ', 'most_common_ancestry']])
```

```
post merge:
Index(['STATEFP', 'COUNTYFP', 'COUNTYNS', 'GEOIDFQ', 'GEOID', 'NAME',
      'NAMELSAD', 'STUSPS', 'STATE_NAME', 'LSAD', 'ALAND', 'AWATER',
      'geometry', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
      'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
      'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
      'Ethnic Chinese', 'Other Southeast Asian', 'most_common_ancestry_raw',
      'most_common_ancestry'],
      dtype='object')
/usr/local/lib/python3.12/dist-packages/pyogrio/geopandas.py:917: UserWarning: 'crs' was r
write(
```

	most_common_ancestry	COUNT	
3	Filipino	479	
2	Ethnic Chinese	341	
9	Vietnamese	60	
6	Korean	49	
4	Hmong	42	
5	Japanese	27	
0	Burmese	20	
7	Laotian	13	
1	Cambodian	4	
8	Thai	3	

<https://gis.stackexchange.com/questions/298530/how-do-i-write-a-geopandas-dataframe-into-a-single-file-preferably-json-or-geop>

<https://geoconverter.mikoding.com/>

https://doc.arcgis.com/en/arcgis-online/manage-data/publish-features.htm#ESRI_SECTION1_49CE0570C3BA4AD8BF2DB28929FF7280

<https://doc.arcgis.com/en/arcgis-online/get-started/print-maps-mv.htm>

ArcGIS mapping steps I used

create new map upload .zip file as base layer upload output.json (GeoJSON file) as 2nd layer (of data)

to change visibility of layers (e.g. changing color intensity, making sure county lines are visible in black)- I clicked on the layer to select it, then the Properties button in the tab on the right (top button), and clicked through the Appearance drop through until I was able to change it to my liking.

2026.01.13 map edits for better visibility after sharing-

<https://pdxedu.maps.arcgis.com/apps/mapviewer/index.html?webmap=a9ffa6184e8e44748fb2fc422e7ed25b>

There's are 2 basemaps available in the ArcGIS map by default- "World Topographic Map" and "World Hillshade". Made both completely transparent to get rid of the green from parks/forests and city labels that detract from the map

cb_2024_us_county_500k layer (feature layer)- set Appearance > Blending to "Normal" and Transparency to 75%

ACS_ESEA_data_output (from GeoJSON)- set Appearance > Blending to "Darken" and Transparency to 25%

PNG files were generated using the ArcGIS Print setting using the following parameters: A4 landscape, PNG32 format, 100 DPI. Scale set manually to 1:25960696